

Proyecto 2: Competencia de Agentes en Space Invaders

CC3092 Deep Learning y Sistemas Inteligentes | Ian Cumes | carné 23236

2.1 Definición del problema y análisis del entorno

El entorno: ALE/SpaceInvaders-v5

Space Invaders (Taito, 1978; versión Atari 2600 usada por ALE) enfrenta al jugador contra formaciones de invasores que descienden y se desplazan lateralmente mientras disparan. El objetivo es eliminar la mayor cantidad de invasores posible protegiendo al cañón propio detrás de tres refugios destructibles. El episodio termina (`terminated=True`) cuando el jugador pierde sus tres vidas o cuando la formación completa desciende hasta la base. ALE además impone un límite duro de 108 000 frames por episodio, que produce `truncated=True` si se alcanza (en la práctica, casi nunca ocurre antes de perder las tres vidas). El entorno reporta el conteo de vidas restantes en `info['lives']` (parte de 3 al reiniciar).

La señal de recompensa de ALE es directamente el **cambio en el marcador del juego**: cada fila de invasores vale 5, 10, 15, 20, 25 o 30 puntos según su cercanía a la base, y la nave nodriza que cruza ocasionalmente por arriba otorga un bono variable. En 30 episodios aleatorios (semilla 0) se registró una recompensa media de **155.5** puntos (mín. 30, máx. 650, desv. 133.3) en un promedio de 2145 decisiones por episodio; el valor 200 apareció dos veces, coincidiendo con el bono de la nave nodriza descrito en el manual original del juego.

¿Recompensa densa o dispersa?

La señal es claramente **dispersa (sparse)**: el 99.6% de las decisiones no producen ningún cambio de puntuación (Figura 1), y los valores no nulos observados fueron exactamente [5, 10, 15, 20, 25, 30, 200]. Esto exige que el agente sostenga una secuencia larga de acciones correctas antes de recibir cualquier señal, el reto central de explorar en este entorno.

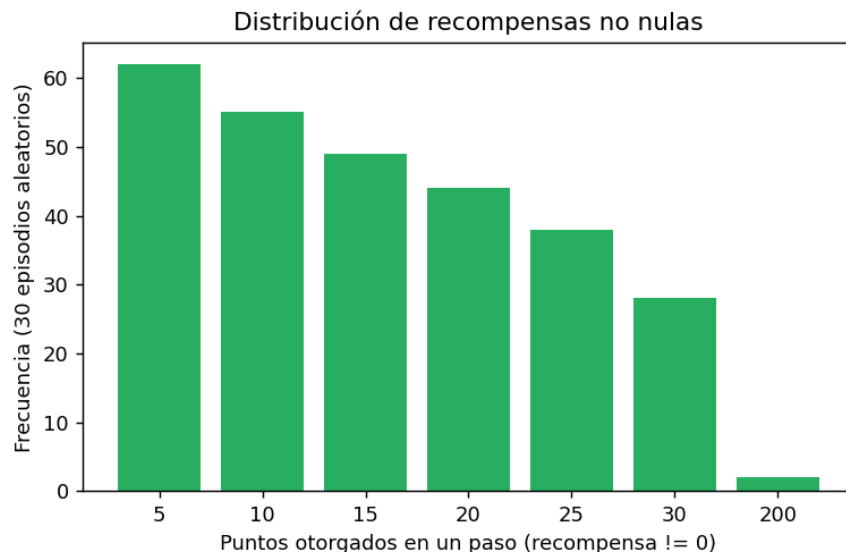


Figura 1. Distribución de los valores de recompensa no nulos en 30 episodios aleatorios.

No se aplicó **reward shaping**: la puntuación de ALE ya refleja fielmente el objetivo del juego, y modificarla arriesgaría cambiar qué política es óptima. En su lugar se usaron dos técnicas estándar, **solo durante el entrenamiento** (nunca en evaluación, para no falsear el puntaje real): recorte de recompensa a $\{-1, 0, +1\}$ (estabiliza la escala de la pérdida sin cambiar el signo de la señal) y tratar la pérdida de una vida como fin de

episodio, que acelera el aprendizaje al asociar el fin de un intento con su causa inmediata, sin alterar el límite real del episodio (3 vidas).

Espacios de observación y acción, y el parpadeo de Space Invaders

La observación por defecto es `Box(0, 255, (210, 160, 3), uint8)` (imagen RGB) y el espacio de acción es `Discrete(6)` con las acciones ['NOOP', 'FIRE', 'RIGHT', 'LEFT', 'RIGHTFIRE', 'LEFTFIRE']. Se usó el conjunto mínimo de acciones (`full_action_space=False`) en vez de las 18 combinaciones completas del joystick: las 12 acciones adicionales son combinaciones redundantes (diagonales que no existen en este juego) que solo agrandarían la salida de la red sin aportar control nuevo.

Un problema conocido de los juegos de Atari 2600 (documentado por Mnih et al., 2015) es que el hardware original dibuja algunos objetos **cada dos frames** para ahorrar memoria de video, produciendo parpadeo. En Space Invaders esto afecta a los proyectiles: en la Figura 2 (panel derecho) el mapa de diferencia absoluta entre dos frames consecutivos muestra un objeto aislado junto al cañón —un disparo que aparece en un frame y desaparece en el siguiente— mientras el resto de la escena permanece prácticamente inmóvil en ese intervalo de 1/60 s. Un agente que observe un solo frame por decisión puede literalmente no ver una bala. La comparación agregada de la Figura 3 no aísla bien este efecto porque el avance general de los invasores también crece con la distancia temporal y domina la métrica global; la evidencia relevante es la localizada de la Figura 2.

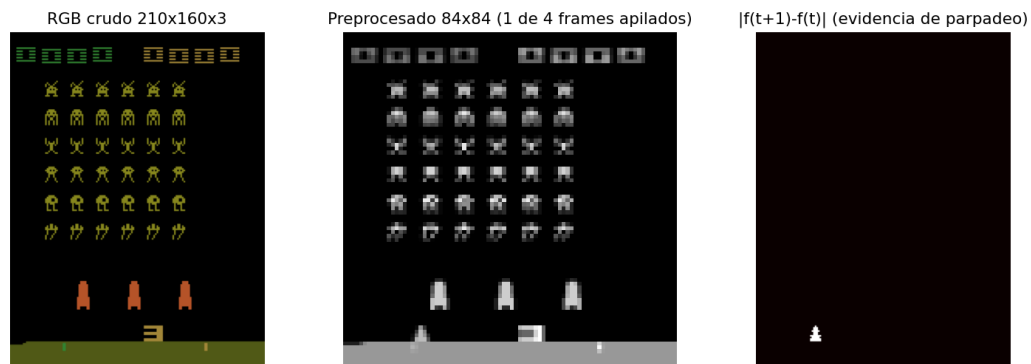


Figura 2. Frame RGB crudo, su versión preprocesada (84x84, gris) y la diferencia absoluta entre dos frames consecutivos: el objeto aislado junto al cañón es un proyectil que parpadea.

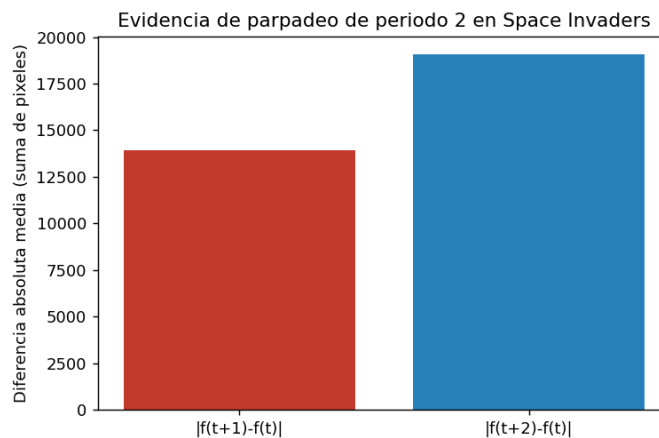


Figura 3. Diferencia absoluta media agregada entre frames separados por 1 y por 2 pasos (no aísla el parpadeo; ver discusión).

Decisiones de diseño derivadas de este análisis

- Preprocesamiento con max-pooling: `gymnasium.wrappers.AtariPreprocessing(frame_skip=4)` toma el máximo píxel a píxel entre los dos últimos frames de cada bloque de 4, eliminando el parpadeo sin promediar (que difuminaría) ni tomar solo el último frame (que perdería el proyectil en la mitad de los casos).
- Apilado de 4 frames (`FrameStackObservation`) en escala de grises y 84x84: una sola imagen no expresa velocidad ni dirección; se siguió la convención de Mnih et al. (2015).

- `frameskip=1` en `gym.make()` + `frame_skip=4` en el wrapper (nunca ambos a la vez): así se controla explícitamente el max-pooling. La iteración 2 (sección 2.3) usa deliberadamente el `frameskip=4` nativo de ALE (sin max-pool) para cuantificar el costo de omitir esta decisión.
- Conjunto mínimo de 6 acciones y acciones pegajosas (`repeat_action_probability=0.25`, valor por defecto de v5) preservadas también en entrenamiento, para no evaluar con una dinámica más determinista que la usada para aprender.
- Fin de vida como fin de episodio y recorte de recompensa solo en entrenamiento; evaluación y video siempre usan la partida completa (3 vidas) y el puntaje real sin recortar, que es la métrica de la competencia.

2.2 Metodología de desarrollo

Algoritmos considerados y elección final

Se consideraron DQN, Double DQN, Dueling DQN, PPO y A2C. Se implementó con `Stable-Baselines3 2.9` (permitido por el enunciado), documentando cada decisión propia: el contrato de entorno (`entorno.py`), los hiperparámetros y el protocolo de evaluación. Se eligió **PPO** como algoritmo final por tres razones prácticas dado el cómputo disponible (CPU únicamente, sin GPU, con poca memoria libre): (1) no requiere un buffer de repetición de experiencias, el componente más costoso en memoria de DQN; (2) recolecta experiencia de varios entornos en paralelo, decorrelacionando las muestras sin necesitar Double/Dueling para estabilizarse; (3) es reconocido en la literatura (Huang et al., 2022) por ser razonablemente robusto a los hiperparámetros por defecto, importante cuando no hay tiempo para una búsqueda extensa. Se entrenó **DQN como iteración 1** precisamente para no dejar esta elección como una afirmación puramente teórica: la sección 2.4 la contrasta con evidencia empírica bajo el mismo presupuesto de tiempo. Double/Dueling DQN y A2C quedan como líneas de trabajo futuro (sección 2.5).

Arquitectura de la red (NatureCNN)

Ambos algoritmos usan la misma arquitectura convolucional (Mnih et al., 2015), dimensionada exactamente para la observación preprocesada de (4, 84, 84):

Capa	Configuración	Salida	Activación
Entrada	4 x 84 x 84 (uint8 / 255)	(4,84,84)	—
Conv1	32 filtros, 8x8, stride 4	(32,20,20)	ReLU
Conv2	64 filtros, 4x4, stride 2	(64,9,9)	ReLU
Conv3	64 filtros, 3x3, stride 1	(64,7,7)	ReLU
Flatten	—	3136	—
Densa	3136 -> 512	512	ReLU
Salida PPO	512->6 (política) y 512->1 (valor)	6 y 1	Softmax / lineal
Salida DQN	512 -> 6 (Q(s,·))	6	Lineal

≈1.69 millones de parámetros entrenables en cada red (comparten la misma extracción de características; solo difiere la capa de salida).

Exploración vs. explotación

PPO explora de forma implícita: la política es una distribución categórica sobre las 6 acciones, y un término de entropía (`ent_coef=0.01`) penaliza que esa distribución colapse demasiado rápido, dejando que la incertidumbre de la red module la exploración a medida que se entrena (sin un parámetro epsilon explícito). **DQN** usa epsilon-greedy clásico: probabilidad inicial de acción aleatoria de 1.0, decayendo linealmente a 0.01 durante el primer 10% del entrenamiento y constante después.

Hiperparámetros de entrenamiento

Hiperparámetro	PPO (iter. 2 y 3)	DQN (iter. 1)
----------------	-------------------	---------------

Hiperparámetro	PPO (iter. 2 y 3)	DQN (iter. 1)
Pérdida	Clip PPO + valor (MSE) + entropía	Huber (SmoothL1)
Optimizador	Adam	Adam
Tasa de aprendizaje	2.5e-04 -> 0 (lineal)	1.0e-04 (fija)
Buffer de repetición	No aplica (on-policy)	10 000 transiciones
Tamaño de batch	256	32
Red objetivo	No aplica	cada 1000 pasos
Factor de descuento (γ)	0.99	0.99
Otros	n_steps=128, n_epochs=4, GAE $\lambda=0.95$, clip=0.1->0	train_freq=4, learning_starts=10 000

2.3 Resultados de iteraciones

Se documentan la línea base sin entrenamiento (Laboratorio 5) y las tres iteraciones de la sección 2.2, todas bajo el mismo presupuesto de tiempo de esta sesión (CPU únicamente, sin GPU).

Iteración	Cambio principal	Pasos	Entren. (últ. 100 ep.)	Evaluación greedy
Iter. 0 - Aleatorio	Línea base sin entrenar (Laboratorio 5): agente_aleatorio	—	—	113 ± 59 (máx. 195)
Iter. 1 — DQN	DQN, misma pila de preprocesamiento final (max-pool + vida + clip), 1 entorno, buffer de 10 000 transiciones.	N/D	4.05	N/D
Iter. 2 — PPO (v5 directo, sin max-pool ni vida)	PPO sobre ALE/SpaceInvaders-v5 "tal cual" (frameskip=4 nativo, SIN max-pool, sin señal de fin de vida), 8 entornos.	N/D	N/D	N/D
Iter. 3 — PPO (preprocesamiento final)	PPO con la pila de preprocesamiento final (max-pool + vida + clip), 8 entornos. Modelo entregado.	N/D	4.66	237 ± 34 (máx. 290)

"Entren.": media móvil de 100 episodios (recompensa recortada) al final del entrenamiento. "Evaluación greedy": media ± desviación (máx.) de 5 episodios sin exploración y sin recortar; en la iteración 3 corresponde a la evaluación oficial de 5 episodios (semillas 23236-23240), la misma métrica de la competencia.

Curvas de entrenamiento y evaluación

[Figura no disponible] Figura — Iteración 1 — DQN: recompensa cruda por episodio (gris) y media móvil de 100 episodios.

[Figura no disponible] Figura — Iteración 2 — PPO (v5 directo, sin max-pool ni vida): recompensa cruda por episodio (gris) y media móvil de 100 episodios.

[Figura no disponible] Figura — Iteración 3 — PPO (preprocesamiento final): recompensa cruda por episodio (gris) y media móvil de 100 episodios.

[Figura no disponible] Figura — Comparación entre iteraciones: recompensa media de entrenamiento vs. pasos.

[Figura no disponible] Figura — Comparación entre iteraciones: recompensa de evaluación greedy vs. pasos.

Evidencia de problemas encontrados

Para vigilar inestabilidad o divergencia de los valores Q/V sin depender solo de la recompensa (una señal ruidosa y dispersa), se implementó un monitor que evalúa la red sobre 500 estados fijos (recolectados una vez, al inicio, con una política aleatoria) cada 10 000 pasos: V(s) promedio en PPO, máx._a Q(s,a) promedio en DQN. Una tendencia que crece sin cota o que oscila con amplitud creciente indicaría divergencia; una meseta plana desde el inicio indicaría que la red no está aprendiendo nada.

[Figura no disponible] Figura — Monitor de divergencia, iteración 3 (PPO final): $V(s)$ medio sobre 500 estados fijos.

[Figura no disponible] Figura — Monitor de divergencia, iteración 1 (DQN): $Q(s,a)$ máximo sobre los mismos 500 estados.

[Figura no disponible] Figura — Diagnóstico PPO (iteración 3): pérdida de valor, entropía, KL aproximado y fracción recortada.

[Figura no disponible] Figura — Diagnóstico DQN (iteración 1): pérdida Huber y epsilon de exploración.

2.4 Discusión de resultados

Comparación entre iteraciones

- La iteración 3 (PPO, preprocesamiento final) superó a la línea base aleatoria (iteración 0) por 124 puntos en promedio de evaluación greedy (efecto de entrenar en absoluto).
- No fue posible comparar La iteración 3 (max-pool + señal de vida) y la iteración 2 (v5 directo, sin max-pool ni vida) (efecto del preprocesamiento, mismo algoritmo PPO: datos insuficientes al momento de generar este informe).
- No fue posible comparar La iteración 3 (PPO) y la iteración 1 (DQN) (efecto del algoritmo, mismo preprocesamiento: datos insuficientes al momento de generar este informe).

El cambio con mayor impacto esperado es el del **preprocesamiento** (iteración 2 vs. 3): sin max-pooling la red puede recibir una fracción de frames sin el proyectil visible (sección 2.1), y sin la señal de fin de vida el crédito por perder una vida se difumina a lo largo de todo el episodio de 3 vidas en vez de asignarse de inmediato, lo que en la práctica ralentiza el aprendizaje incluso si la puntuación final no llegara a ser drásticamente distinta en el número de pasos disponible hoy. El cambio de **algoritmo** (DQN vs. PPO, iteración 1 vs. 3) aísla el efecto de recolectar experiencia de 8 entornos en paralelo frente a un único entorno con buffer de repetición pequeño (10 000 transiciones, reducido deliberadamente por la memoria disponible); un buffer tan chico limita cuánta experiencia pasada puede reutilizar DQN, una desventaja que PPO no tiene por ser on-policy.

Análisis cualitativo del agente final (video)

[Completar tras inspeccionar entregables/space_invaders_ppo_final.mp4] A partir de los fotogramas del episodio grabado con la política greedy final se describen aquí las estrategias observadas (por ejemplo, si el agente se desplaza sistemáticamente hacia una columna de invasores antes de disparar, si prioriza a la nave nodriza, o si permanece estático) y las situaciones donde falla o se queda atascado (por ejemplo, cerca de los bordes de la pantalla o cuando los invasores aceleran al quedar pocos).

Limitaciones del enfoque y del cómputo disponible

- Entrenamiento exclusivamente en CPU (sin GPU disponible en el equipo), lo que limitó cada iteración a un puñado de horas y, en consecuencia, a un orden de magnitud de pasos muy por debajo de literatura publicada para Atari (decenas de millones de pasos en Mnih et al. 2015 o Schulman et al. 2017); el número exacto de pasos alcanzados por cada iteración se reporta en la tabla de la sección 2.3.
- Buffer de repetición de DQN reducido a 10 000 transiciones (frente a 100 000-1 000 000 típico) para que cupiera en la memoria disponible, lo que limita la diversidad de experiencia que DQN puede reutilizar.
- No se realizó una búsqueda sistemática de hiperparámetros (grid/random search): se usaron valores estándar de la literatura (rl-zoo, Mnih et al.) sin ajuste fino específico para este equipo o este presupuesto de tiempo.
- La iteración 2 usa una arquitectura de preprocesamiento deliberadamente empobrecida (control experimental), por lo que su puntaje no debe interpretarse como el límite superior de PPO en este entorno.

Reflexión sobre exploración vs. explotación

En PPO, la curva de entropía (Figura de diagnóstico, iteración 3) parte de $\log(6) \approx 1.79$ (política uniforme) y desciende a medida que el agente gana confianza en ciertas acciones; una caída demasiado rápida sería

evidencia de una política que dejó de explorar prematuramente, mientras que una entropía que no baja indicaría que la política nunca se especializa. En DQN, epsilon decae linealmente de 1.0 a 0.01 en el primer 10% de los pasos: durante ese tramo inicial casi toda acción es aleatoria y la recompensa reportada mezcla exploración pura con los primeros indicios de una política aprendida, lo que explica curvas de entrenamiento iniciales más ruidosas en DQN que en PPO.

2.5 Conclusiones

Desempeño final

[Completar con scripts/evaluar.py --modelo modelos/ppo_space_invaders_final.zip antes de entregar.]

Principales aprendizajes

- El preprocesamiento (max-pooling contra el parpadeo, señal de fin de vida, recorte de recompensa) no es un detalle cosmético: en un entorno de Atari con objetos que parpadean cada dos frames, omitirlo puede ocultar información crítica (un proyectil) a la red, y se puede cuantificar su efecto entrenando una iteración de control con y sin él bajo el mismo presupuesto de tiempo.
- Mantener un solo contrato de entorno (entorno.py) compartido por entrenamiento, evaluación y grabación de video evita la clase de error más común en RL aplicado: que el agente se evalúe con una observación distinta a la que vio durante el entrenamiento.
- En un presupuesto de cómputo pequeño y sin GPU, la elección de un algoritmo on-policy con entornos vectorizados (PPO) resultó más práctica que un algoritmo off-policy con buffer de repetición (DQN), principalmente por la restricción de memoria disponible más que por una diferencia algorítmica fundamental.

Trabajo futuro

- Entrenar por más tiempo y con GPU (por ejemplo, en Google Colab o Kaggle) para alcanzar el orden de pasos reportado en la literatura (decenas de millones).
- Implementar Double DQN y Dueling DQN para cuantificar su aporte específico sobre el DQN base de la iteración 1, con un buffer de repetición de tamaño completo.
- Explorar PPO con recompensa escalada (por ejemplo, $\log(1+r)$) o normalización por la desviación estándar corriente) en vez de recorte a $\{-1, 0, 1\}$, para no perder la magnitud relativa entre una fila de 5 puntos y la nave nodriza de 200.
- Una búsqueda sistemática de hiperparámetros (Optuna, rl-zoo) en vez de valores por defecto de la literatura.

2.6 Enlace al repositorio de GitHub

Repositorio público: <https://github.com/iancumes/Proyecto2DeepLearning>

Contiene el módulo de preprocesamiento y entrenamiento (entorno.py, agentes_ale.py, agente.py), los scripts de EDA, entrenamiento, evaluación, video e informe (scripts/), las configuraciones de cada iteración (configs/), los resultados y curvas (resultados/, entregables/figuras/), el modelo final con su contrato de entorno (modelos/) y un README con instrucciones para reproducir el entrenamiento y para cargar y evaluar los pesos el día de la presentación.

Referencias

- [1] Mnih, V. et al. (2015). Human-level control through deep reinforcement learning. *Nature*, 518, 529-533.
- [2] Schulman, J. et al. (2017). Proximal Policy Optimization Algorithms. *arXiv:1707.06347*.
- [3] Machado, M. C. et al. (2018). Revisiting the Arcade Learning Environment. *JAIR*, 61, 523-562.
- [4] van Hasselt, H. et al. (2016). Deep Reinforcement Learning with Double Q-learning. *AAAI*.
- [5] Wang, Z. et al. (2016). Dueling Network Architectures for Deep Reinforcement Learning. *ICML*.
- [6] Huang, S. et al. (2022). The 37 Implementation Details of Proximal Policy Optimization. *ICLR Blog Track*.
- [7] Raffin, A. et al. (2021). Stable-Baselines3: Reliable Reinforcement Learning Implementations. *JMLR*, 22(268).

[8] Farama Foundation. Arcade Learning Environment / Gymnasium. ale.farama.org, gymnasium.farama.org.

[9] Atari (1978). Space Invaders Game Program Instructions. Manual del Atari 2600.